

AI RESEARCH PAPER (Updated).docx

Facial Recognition System using Transfer Learning with the help of VGG16

Rajnishkumar Mishra¹, Saeed Wadekar², Suraj Warbhe³, Sayali Dalal⁴, Riddhi Mirajkar⁵

^{1,2,3,4,5} ¹⁶ Department of Information Technology and Engineering; Vishwakarma Institute of Information Technology, Pune, India

Abstract-

²² Facial recognition is one of the most important and widely studied topics in the field of Computer Vision and Artificial Intelligence. The ability of automation to detect and verify a person's face using photography is a very important factor in monitoring, device access control (Laptop and mobiles), security, tracking, law enforcement, biometrics, Information security, smart cards, surveillance systems, and other similar fields. Various applications help to identify specific people in specific areas that help to find intruders. Real-time recognition is an obligation for surveillance purposes. This research paper introduces an optimized and efficient facial recognition system with a very advanced face recognition algorithm. This proposed methodology is processed into five main stages: (i) Data Collection (ii) Data Cleaning (iii) Fine-tuning of the VGG16 model and (iv) Model Training, and (v) Performance Checking. The first step is collecting face images of different people. In the second step, face images are cropped and saved to remove unnecessary, irrelevant, or meaningless data (noise). In the third step, a very popular image recognition model VGG16 along with is fine-tuned for the specific case using CNN (Convolutional Neural Network). The proposed work is implemented using a python module named Keras. Lastly, performance is measured using a testing dataset that is different from the training dataset.

Keywords: Artificial Neural Network, Deep learning (DL), Convolutional Neural Networks (CNNs), VGG16, Face Recognition, Computer Vision (CV), Big Data, AlexNet, ResNet

1. INTRODUCTION

⁶ Facial detection and recognition is a very vital topic studied in the field of CV and Artificial Intelligence. Nowadays, applications of Facial Recognition are widespread everywhere due to its feasibility to use, efficiency, and emergence in the past few years. The fascinating challenges & advancements in Facial Recognition Systems have been attracting many engineers, data analysts, and data scientists working in Pattern Recognition, Computer Vision, and biometrics fields. Various facial recognition algorithms are being used in various programs such as compression video and indexing that come under the domain of biometrics. One of the common uses of biometrics is unlocking phones using facial recognition. It's very much popular due to its ease and convenience to use when

¹ ²¹ this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

compared to other unlocking methods like password, pin, pattern, fingerprint, iris scanner, etc. Facial recognition plays a very vital role in biometrics nowadays. Facial recognition techniques are also used to classify contents of multimedia and to assist in a faster search of items of interest to the end-user. A well-trained facial recognition is widely used in areas such as forensic science and surveillance. It can also be used by law enforcement authorities for enhanced security, and by banking systems. Facial recognition systems have become important after the rise of terrorism and crimes in the last few years. In any case, facial recognition should be used along with other safety and security measures. Face recognition systems are being used at many significant places for security like at airports, museums, etc. And the demand is expected to grow exponentially in the coming years.

Despite the rapid growth in the field of Facial Recognition as an important validation method, the algorithms used to detect and identify Human faces have neither been optimized nor improved yet. It is 25+ years since facial recognition systems are being developed but any system has not been able to give desired results in real-life and real-time conditions yet. NIST's (National Institute of Standards and Technology) FRTV (Face Recognition Vendor Test) has shown that the facial recognition methods available don't work well under various real-life conditions.

Modern facial recognition programs aimed at complex areas are becoming very crucial in recent decades. Facial recognition systems are integrated and automated with emerging technologies that have gained very much attention. Color images increase the data complexity as the pixels are mapped into a larger size. This greatly increases the need for computing resources and time while the accuracy of the model decreases. So, the higher the image resolution is, the more difficult it is to deal with. In the past few years, it has been suggested that in-depth reading works best when you have BigData. On the contrary, it has also been recognized that conventional ML methods may work better on comparatively small databases.

The proposed Face Recognition System analyzes color images so that faces can be detected and identified accurately. CNN along with VGG-16 has been used in the proposed study to get improved accuracy along with reducing the amount of time and resources spent. The biggest challenge for a facial recognition system is the recognition and rendering of a feature (i.e data cleaning). The algorithm that has been suggested here uses in-depth reading strategies to extract features. The proposed algorithm uses in-depth reading to detect faces accurately. The proposed algorithm will be able to detect faces accurately so it can be used to search and identify suspects.

• 2. LITERATURE REVIEW

In this section, various types of facial recognition approaches based on Convolutional Neural networks, Machine Learning, Artificial Neural networks, Deep Learning, and different facial algorithms are discussed.

Savath Saypadith and Supavadee Aramvith [1] proposed a new approach to facial recognition based on the Convolutional and Deep Convolutional Neural Network facial recognition algorithm. This model has been demonstrated on the embedded Graphics Processing Unit system. In 0.23 seconds, this system can recognize 8 faces simultaneously with an accuracy of above 83.67%.

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

Chetna Singh, Sarvesh Singh, and Prashant Baheti [2] proposed a Skin Colour Model that combined PCA (Principal Component Analysis) and YCbCr for Human Face Detection and Recognition and Detection. This algorithm is capable of detecting and recognizing human faces from an image. This model functions well for Indian faces as they have a little variation in skin texture, face complexion, and color. The PCA for face recognition and YCbCr for face detection are used and have shown effective results with an overall success rate of 95%.

Shubha P. Jagi and M. Meenakshi [3] proposed a model based on the Local Binary Pattern algorithm and Image Processing approach that has been implemented in real-time Face recognition. This algorithm is demonstrated on MATLAB software. This model has achieved an accuracy of 89% by which it can be concluded that the Local Binary Pattern Algorithm is better than Principal Component Analysis (PCA), 2D-PCA, and LDA.

KH Teoh, SZM Naziri, MNM Isa, RC Ismail, MSSM Basir, and R Hussin [4] proposed a model for Face Detection and Recognition using Haar feature-based cascade classifiers and the Convolutional Neural Network approach. This model used Haar Cascade for the frontal face and the training is generally done on a server and various stages. This Haar Cascade-based model has shown effective results with an overall success rate of 91.7% in recognizing images and 86.7% in real-time video.

Suleman Khan, M. Hammad Javed, Ehtasham Ahmed, Syed Umaid Ali, and Syed A A Shah [5] proposed a method to classify almost 1000 different people and identify their faces in an image or a real-time video. The proposed methodology is based on the approach of Transfer Learning. This model has been demonstrated on MATLAB Software using Haar-Cascade Classifiers and trained using the AlexNet Model of Neural Network which shows a success rate of 97.95%.

Narayan T. Deshpande and Dr. S. Ravishankar [6] proposed a method for human face detection and recognition. The developed methodology used the "BioID Face Database" as the standard image database. This method is efficient and is not dependent on distinct facial features like facial emotions, facial gestures, and expressions, face texture, face color and complexion, change in hairstyle, etc using the of Viola-Jones algorithm, PCA (Principal Component Analysis), and Artificial Neural Network(ANN) and achieved an accuracy of 94%.

Yi Sun, Xiaogang Wang, Ding Liang, and Xiaoou Tang [7] proposed a Face Recognition model named "DeepID3". DeepID3 is a combination of two different types of DNN (Deep Neural Network) Architectures i.e inception layers and stacked convolution. This stacked convolution is proposed in GoogleNet Model and specifically planned for face recognition. This DeepID3 model achieved a success rate of 99.53% for "Labeled Faces in the Wild (LFW)" face verification.

Manik Sharma, G S C Kashyap, H KManne, and J Anuradha [8] proposed a model based on Deep Belief Network (DBN). This developed model recognizes faces in still images and paintings, real-time videos, the captured images of webcam, blurred images, and side-faced images. In this, Deep Belief Network creates a layer-by-layer architecture that primarily consists of an input layer and an output layer, and a number of different layers are hidden between the input and output layer. This layered architecture should be trained with Restricted Boltzmann Machines (RBMs) to reconstruct input data. By combining all RBMs, they have introduced a collaborating method to get a powerful new model.

3

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

Deepesh Raj [9] proposed a model of unsupervised learning (i.e. Principal Component Analysis (PCA) Algorithm) for instantaneous face recognition. This developed methodology majorly consists of three different stages: Acquisition, Feature Extraction, and Face Recognition. The implementation of the proposed system was done on C++ and OpenCV Library whereas Feature Extraction was performed using PCA Algorithm and three different types of distance Classifiers i.e. Euclidean distance, Mahalanobis distance, and Manhattan Distance. This proposed model achieved a success rate of face recognition of about 92.3% whereas the normal Principal Component Analysis (PCA) Algorithm achieved a success rate of 73.1%. This trained model estimated the single queries of face recognition in less than 0.2 seconds.

Showkat A. Dara, S.Palanivel [10] introduced a new methodology to perform operations of face detection and face recognition systems using the approach of the VGG16 Neural Network Model. The proposed methodology is implemented in the Tensorflow Library (Keras). It is proposed on the approach of Convolutional Neural Network and Transfer Learning and achieved a success rate of 99.37%.

S. SwethaK, E. Surekha, S. Sindhuja, and Rohini Kailas [11] proposed a methodology of real-time automated attendance systems for students and employees. The proposed system was developed for organizations such as colleges or small startups to assist them in the basic day-to-day activities. It is proposed on the approach Haar Cascade Classifiers and Local Binary Patterns Algorithm and focuses on capturing images from a live video stream and crediting attendance based on recognition of faces in the image. The main function of the system was to enroll the subject's face against the subject's ID (unique) and Name in the database. It also helped in maintaining attendance records by allotting attendance to the recognized subjects' faces in the database.

B. Ramesh, H. C. Impana, P. Apoorva, S. Siri, and M. Varshitha. [12] had proposed the use of surveillance cameras for face recognition using a Haar classifier. The proposed system could successfully recognize multiple faces at once without much computation time. The proposed system consisted of four steps- (i) Training (ii) Detection and Recognition of faces using the Haar-classifier (iii) Comparison between surveillance camera images and trained real-time images (4) Comparison based Result

K. Jaya Prasanna Lakshmi, Sridevi Warriar, and T. Kumar [13] had proposed an automated Smart Security System based on facial recognition using AI/ML algorithms. The proposed system could detect an intruder through surveillance camera based on facial recognition using AI/ML algorithms. The specified design was implemented with the help of HOG(Histogram of Oriented Gradients) feature extraction and SVM (Support Vector Machine) classifier which takes a video stream as an input and classifies the faces.

Andrea Vedaldi, Omkar M. Parkhi, and Andrew Zisserman [14] proposed a facial recognition method from either a single image or from a set of images captured in a video. One of the aims of their project was to show how a very large dataset of 2.6 million images over 2600 people could be collected using the combination of computer automation and humans. Another aim was to study the different complexities of facial recognition using deep neural networks. And present the methods to achieve comparable state-of-the-art results concerning LFW and YTF face benchmarks on the database.

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

3. PROPOSED METHODOLOGY

The proposed methodology can be divided into five sequential major steps:-

- i) Data Collection
- ii) Data Cleaning
- iii) Fine-tuning of VGG16 model
- (iv) Model Training, and
- (v) Performance Checking.

The first step is a collection of face images of different people. Collection of images has been majorly done through Webcam which is further used as a training and testing dataset. In the second step, collected face images are cropped to specific dimensions and saved to remove unnecessary, irrelevant, or meaningless data (noise), remove noise. In the third step, a very popular image recognition Neural Network model VGG16 with is fine-tuned for the specific case with the help of a Convolutional Neural Network. The proposed work is implemented using a python module named Keras. Lastly, performance is measured using a test dataset that is different from the training dataset. The proposed system's building block can be seen in Figure 1.

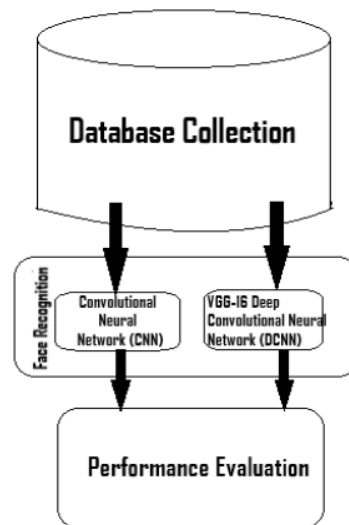


FIGURE 1: BUILDING BLOCK OF FACIAL RECOGNITION SYSTEM

CNN(Convolutional Neural Network):

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

20

Convolutional Neural Network also commonly known as CNN, is one of the most powerful deep neural networks. Raw pixel of images acts as data for CNN, the model is trained, then the features are extracted automatically for better classification. It was a revolution as it was very difficult to create a system that would understand visual data. Ever since CNN has played a very important role in the operations of CV. An input layer following multiple hidden layers followed by an output layer is the architecture of convolutional networks. There are various layers in the CNN model which are shown in Figure 2: Input layer, Convolutional Layer, Pooling Layer, ReLU correction layer, Fully-connected layer, and Output layer.

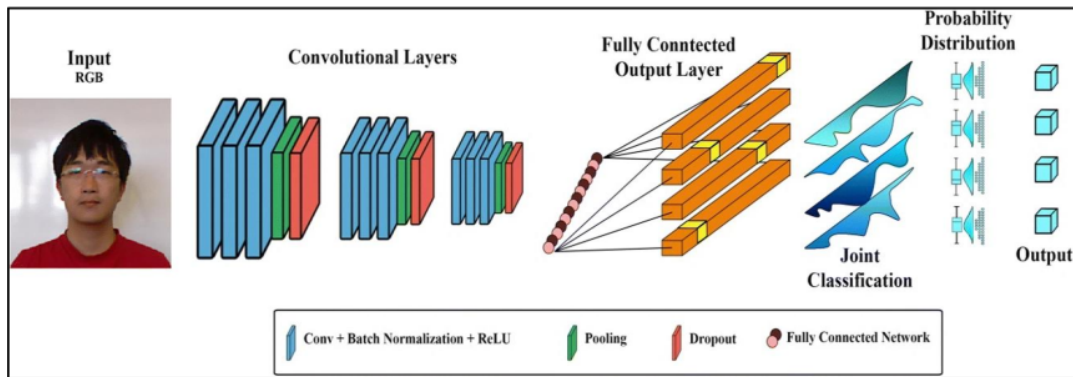


FIGURE 2: CNN ARCHITECTURE (Source: Google)

- **Convolution Layer -**

It is the first and most critical layer in CNN. Its purpose is to detect the features present in the images received as input. It receives the image dataset as input and calculates its convolution for each filter. Filters get the same features we want to get in photos. Features are not predetermined but are learned during the training phase.

- **Pooling Layer -**

This layer lies between 2 Convolution + ReLU layers. It finds various feature maps and pooling operations are applied to each of them. The pooling operation involves preserving features while reducing image size. The main function of this layer is to reduce the number of parameters and reduce calculations. This results in increased efficiency and reduces the chance of over-learning. Different types of pooling functions are shown in Figure 3.

6

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

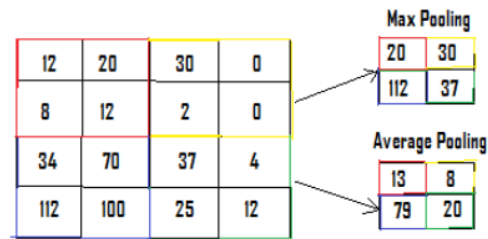


FIGURE 3. POOLING FUNCTION

- **ReLU Correction Layer-**

18

ReLU is also known as Rectified Linear Unit and acts as an activation function. It is defined by $ReLU(x) = \max(0, x)$. It is a piecewise linear function when the input is positive, the output is the same as the input and in all other cases output remains zero. The function is shown in Figure 4.

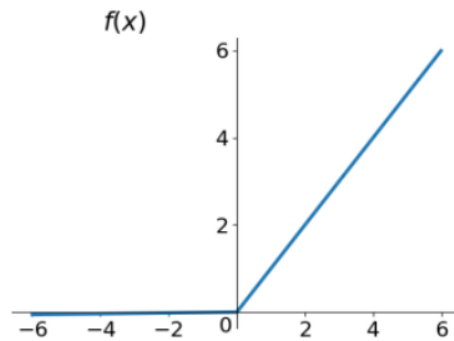


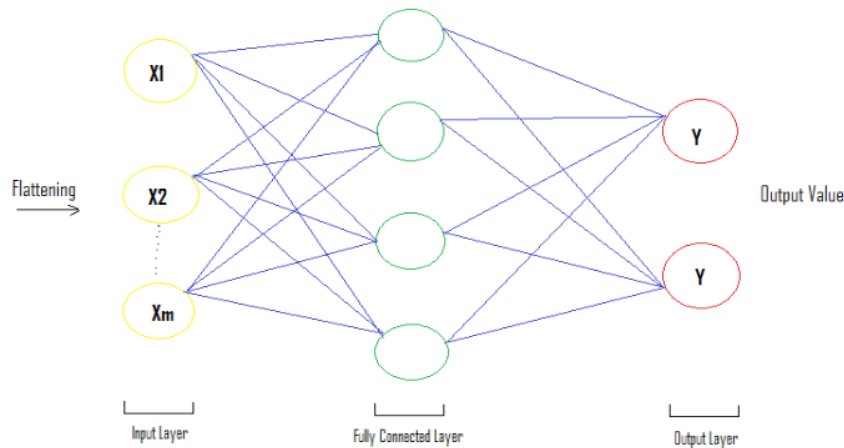
FIGURE 4. RELU ACTIVATION FUNCTION

- **Fully Connected Layer -**

It always remains the last layer of any neural network. The job of this layer is to receive an input vector which as a result produces a new output layer that separates the image as a network input: and returns the N-size vector, where N is equal to the number of classes in the image separation problem. Each vector element indicates the probability that the inserted image belongs to a class.

7

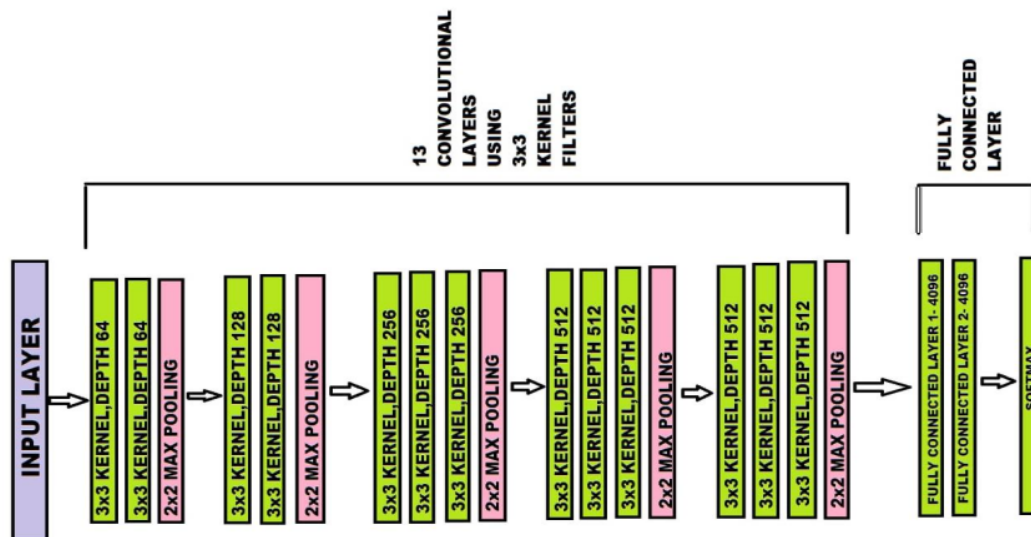
In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.



27
FIGURE 5. FULLY CONNECTED LAYER IN CNN

VGG-16 Neural Network Model

VGG16 is one of the modular Convolutional Neural Network models proposed in the paper "Extensive Convolution Networks for Great Image Recognition". It was one of the most popular models submitted in ILSVRC-2014. The model achieved a success rate of 92.7% in almost 5 ImageNet, which is 14 million photographic sites of 1000 classes. Accuracy will be increased by changing it with large kernel-sized filters (11 & 5 in the first and second convolution layers, respectively) with multiple kernel-sized filters 3×3 respectively. NVIDIA Titan Black GPUs were used to train VGG16 for weeks.



In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

FIGURE 6: VGG16 ARCHITECTURE (Source: Google)

Cov1 layer input is generally a static RGB image of dimension 224×224 . This image is transferred to a series of convolutional layers, where different filters are applied with the smallest receiving field: 3×3 (which is the smallest imaging left / right, top/bottom, center.). In one setting, it also uses 1×1 dimensional convolution filters as input channels (followed by non-linear). A convolution action is limited to only 1 pixel; local padding convolutional layer input is similar to the location adjustment saved after conversion, i.e. padding ranges from 1 pixel to 3×3 convolutional layers. Consolidation of space is made up of 5 major layers of sequencing, followed by other convolutional layers (not all the layers of convolution are followed by multiple combinations). Max-pooling is performed with a window of size 2×2 -pixel, in 2 steps.

There are three Fully Connected (FC) layers that follow a series of some flexible layers (depths vary with structures): the first two fully connected layers have around 4096 channels each and the third fully connected layer developed a division of ILSVRC which is of around 1000 channels (one channel for each class). The softmax layer is the last fully connected layer. Fully integrated layouts are the same across all networks. (see Figure 6)

Transfer Learning

Transfer learning helps to improve learning performance and reduce computation cost and time to a great extent. The main idea of transfer learning is to borrow labeled facial images to achieve great performance in a particular area of interest. Transfer Learning takes out the data or information from previous source domain functions to identify new target domain tasks from the pre-trained facial images. In transfer learning, the knowledge of an already trained model is used to solve a different but relatable problem. (See Figure 7).

Transfer Learning is used in this proposed methodology because training any model takes a lot of time and computing resources and in this fast-paced world, it's not possible to spend very much time on a particular thing. As we are already aware that we have to follow hit and trial methodology for finding the right hyperparameters. For that, we have to train a model again and again for different combinations of hyperparameters and check which one provides the best result. So, to avoid that we use a pre-trained model attach some new layers, and fine-tune some existing parameters. Using this we don't have to train the whole model, again and again, we just have to train the new and modified layers. This helps in saving a lot of time and computing resources.

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

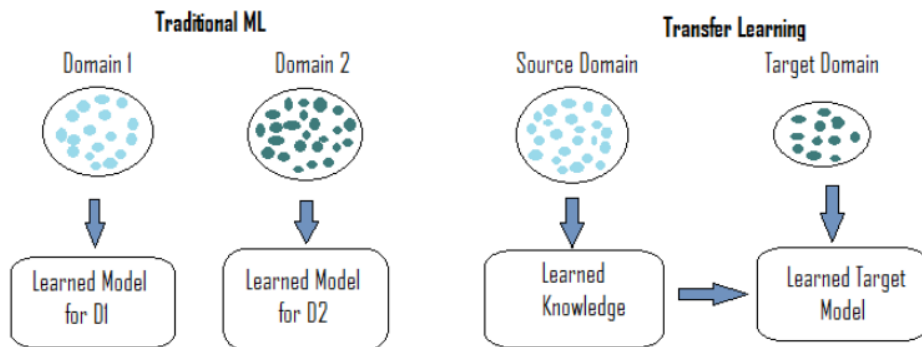


FIGURE 7: TRADITIONAL VS.TRANSFER LEARNING

- **Dataset Collection and cleaning**

Data can be collected using two ways: manually or by using a script. The manual method for data collection is very time taking and resource-intensive as one has to manually collect data, remove noise, and format all the image data of the same size or dimensions. So instead, a script is used for data collection. It uses the *Haar Cascade model* to detect the face and the area in which the face is present is captured and saved. This saves the majority of the time. The code for this part can be found in Figure 8.

```
# Initialize Webcam
import cv2
import numpy as np

# Load HAAR face classifier
face_classifier = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

# Load functions
def face_extractor(img):
    # Function detects faces and returns the cropped face
    # If no face detected, it returns the input image

    faces = face_classifier.detectMultiScale(img, 1.3, 5)

    if faces is ():
        return None

    # Crop all faces found
    for (x,y,w,h) in faces:
        cropped_face = img[y:y+h, x:x+w]

    return cropped_face
cap = cv2.VideoCapture(0)
count = 0
```

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

```

# Collect 100 samples of your face from webcam input
while True:

    ret, frame = cap.read()
    if face_extractor(frame) is not None:
        count += 1
        face = cv2.resize(face_extractor(frame), (224, 224))

        # Save file in specified directory with unique name
        file_name_path = 'C://Users//Rajnish - The Great//Desktop//rajnish' + str(count) + '.jpg'
        cv2.imwrite(file_name_path, face)

        # Put count on images and display live count
        cv2.putText(face, str(count), (50, 50), cv2.FONT_HERSHEY_COMPLEX, 1, (0, 255, 0), 2)
        cv2.imshow('Face Cropper', face)

    else:
        print("Face not found")
        pass

    if cv2.waitKey(1) == 13 or count == 200: #13 is the Enter Key
        break

cap.release()
cv2.destroyAllWindows()
print("Collecting Samples complete")

```

FIGURE 8: DATA COLLECTION AND CLEANING

- **Loading the VGG16 model and fine-tuning**

The VGG16 model is loaded and all the existing layers are frozen so they don't train again. We add a few layers and fine-tune VGG16 to make this model customized for this specific use case. The last layer is the softmax layer to get the output in the desired class. The code for this part can be found in Figure 9.

Face Recognition with VGG16

Loading the VGG16 Model

```

In [1]: from keras.applications import VGG16

# VGG16 was designed to work on 224 x 224 pixel input images sizes
img_rows = 224
img_cols = 224

# loads the VGG16 model
model = VGG16(weights = 'imagenet',
               include_top = False,
               input_shape = (img_rows, img_cols, 3))

```

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

```

In [3]: from keras.applications import VGG16

# VGG16 was designed to work on 224 x 224 pixel input images sizes
img_rows = 224
img_cols = 224

# Re-Loads the VGG16 model without the top or FC layers
model = VGG16(weights = 'imagenet',
               include_top = False,
               input_shape = (img_rows, img_cols, 3))

# Here we freeze all the layers of pre-trained model
# Layers are set to trainable as True by default
for layer in model.layers:
    layer.trainable = False

# Let's print our layers
for (i, layer) in enumerate(model.layers):
    print(str(i) + " " + layer.__class__.__name__ + " " + layer.trainable)

0 InputLayer False
1 Conv2D False
2 Conv2D False
3 MaxPooling2D False
4 Conv2D False
5 Conv2D False
6 MaxPooling2D False
7 Conv2D False
8 Conv2D False
9 Conv2D False
10 MaxPooling2D False
11 Conv2D False
12 Conv2D False
13 Conv2D False
14 MaxPooling2D False

```

```

In [4]: def addTopModel(bottom_model, num_classes, D=256):
    """creates the top or head of the model that will be
    placed ontop of the bottom layers"""
    top_model = bottom_model.output
    top_model = Flatten(name = "flatten")(top_model)
    top_model = Dense(D, activation = "relu")(top_model)
    top_model = Dropout(0.3)(top_model)
    top_model = Dense(num_classes, activation = "softmax")(top_model)
    return top_model

```

```

In [7]: from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Conv2D, MaxPooling2D, ZeroPadding2D
from keras.layers.normalization import BatchNormalization
from keras.models import Model

num_classes = 2

FC_Head = addTopModel(model, num_classes)

modelnew = Model(inputs=model.input, outputs=FC_Head)

print(modelnew.summary())

```

FIGURE 9: VGG16 AND FINE-TUNING

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

- **Loading dataset and Training Model**

As for training a model, we need Big Data so we can get accurate results in all conditions. But here we have very few amounts of data, so we apply various filters to increase the dataset. The code for this part can be found in Figure 10.

```
In [11]: from keras.preprocessing.image import ImageDataGenerator

train_data_dir = 'D://face//train//'
validation_data_dir = 'D://face//validation//'

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')

validation_datagen = ImageDataGenerator(rescale=1./255)

# Change the batchsize according to your system RAM
train_batchsize = 64
val_batchsize = 64

train_generator = train_datagen.flow_from_directory(
    train_data_dir,
    target_size=(img_rows, img_cols),
    batch_size=train_batchsize,
    class_mode='categorical')

validation_generator = validation_datagen.flow_from_directory(
    validation_data_dir,
    target_size=(img_rows, img_cols),
    batch_size=val_batchsize,
    class_mode='categorical',
    shuffle=False)

Found 544 images belonging to 2 classes.
Found 200 images belonging to 2 classes.
```

```
In [9]: from keras.optimizers import RMSprop
from keras.callbacks import ModelCheckpoint, EarlyStopping

checkpoint = ModelCheckpoint("facemodel.h5",
                             monitor="val_loss",
                             mode="min",
                             save_best_only = True,
                             verbose=1)

earlystop = EarlyStopping(monitor = 'val_loss',
                           min_delta = 0,
                           patience = 3,
                           verbose = 1,
                           restore_best_weights = True)

# we put our call backs into a callback list
callbacks = [earlystop, checkpoint]

# Note we use a very small learning rate
modelnew.compile(loss = 'categorical_crossentropy',
                 optimizer = RMSprop(lr = 0.001),
                 metrics = ['accuracy'])

nb_train_samples = 544
nb_validation_samples = 200
epochs = 1
batch_size = 64
```

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.


```

history = modelnew.fit_generator(
    train_generator,
    steps_per_epoch = nb_train_samples // batch_size,
    epochs = epochs,
    callbacks = callbacks,
    validation_data = validation_generator,
    validation_steps = nb_validation_samples // batch_size)

modelnew.save("facemodel.h5")

Epoch 1/2
8/8 [=====] - ETA: 0s - loss: 2.9264 - accuracy: 0.8500
Epoch 00001: val_loss improved from inf to 0.00060, saving model to facemodel.h5
8/8 [=====] - 160s 20s/step - loss: 2.9264 - accuracy: 0.8500 - val_loss: 5.9737e-04 - val_accuracy: 1.0000
Epoch 2/2
8/8 [=====] - ETA: 0s - loss: 0.0166 - accuracy: 0.9917
Epoch 00002: val_loss did not improve from 0.00060
8/8 [=====] - 206s 26s/step - loss: 0.0166 - accuracy: 0.9917 - val_loss: 0.0047 - val_accuracy: 1.0000

```

FIGURE 10: LOADING DATASET AND TRAINING MODEL

- Validation and Prediction

```

In [1]: from keras.models import load_model

classifier = load_model('facemodel.h5')

Testing our classifier on some test images

In [*]: import os
import cv2
import numpy as np
from os import listdir
from os.path import isfile, join

face_data_dict = {"[0]": "Rajnish",
                  "[1]": "Suraj"}

face_data_dict_n = {"rajnish": "Rajnish",
                    "suraj": "Suraj"}

def draw_test(name, pred, im):
    monkey = face_data_dict[str(pred)]
    BLACK = [0,0,0]
    expanded_image = cv2.copyMakeBorder(im, 80, 0, 0, 100, cv2.BORDER_CONSTANT, value=BLACK)
    cv2.putText(expanded_image, monkey, (20, 60), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)
    cv2.imshow(name, expanded_image)

```

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

```

def getRandomImage(path):
    """function loads a random images from a random folder in our test path """
    folders = list(filter(lambda x: os.path.isdir(os.path.join(path, x)), os.listdir(path)))
    random_directory = np.random.randint(0, len(folders))
    path_class = folders[random_directory]
    print("Class - " + face_data_dict_n[str(path_class)])
    file_path = path + path_class
    file_names = [f for f in listdir(file_path) if.isfile(join(file_path, f))]
    random_file_index = np.random.randint(0, len(file_names))
    image_name = file_names[random_file_index]
    return cv2.imread(file_path+"/"+image_name)

for i in range(0,10):
    input_im = getRandomImage("D://face//validation//")
    input_original = input_im.copy()
    input_original = cv2.resize(input_original, None, fx=0.5, fy=0.5, interpolation = cv2.INTER_LINEAR)

    input_im = cv2.resize(input_im, (224, 224), interpolation = cv2.INTER_LINEAR)
    input_im = input_im / 255.
    input_im = input_im.reshape(1,224,224,3)

    # Get Prediction
    res = np.argmax(classifier.predict(input_im, 1, verbose = 0), axis=1)

    # Show image with predicted class
    draw_test("Prediction", res, input_original)
    cv2.waitKey(0)

cv2.destroyAllWindows()

```

FIGURE 11: VALIDATION AND PREDICTION

- OUTPUT

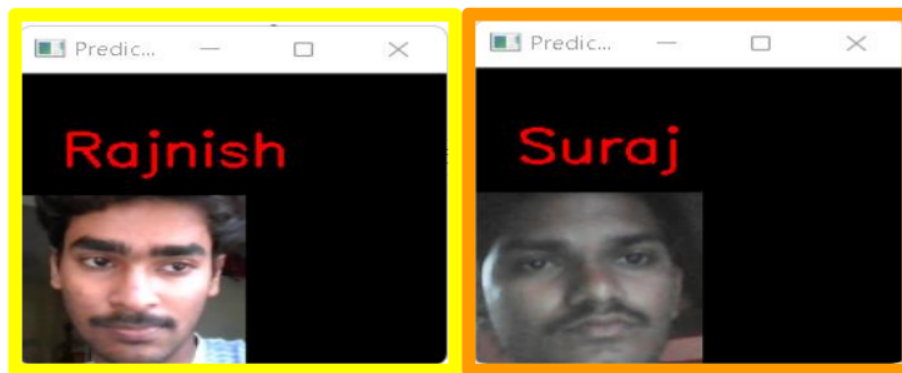


FIGURE 12: OUTPUT

-
- 4. RESULT AND DISCUSSION

This section mainly focuses on how we can get an accuracy score for the trained model. It also discusses the comparison between two different facial recognition algorithms i.e. AlexNet and VGG16 Neural Network Model.

The accuracy of a model can be determined with the help of a confusion matrix. It's used to describe the accuracy of a classification model. The confusion matrix is not only limited to binary classification and can be used in multi-classification. As facial recognition is also a type of classification (generally multi), we can use a confusion matrix to determine the accuracy. The values of the

15

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

confusion matrix need to be calculated for each class so that the i th confusion matrix considers the g_{ia} phase considered to be positive and the remaining categories g_j by $j \neq i$ as negative. Since each confusion matrix combines all the visuals labeled with a different category except g_i as a negative category, this method increases the number of real negatives. This gives us:

- 1) When the event values are predicted correct, it is True Positive (TP)
- 2) When the event values are predicted incorrect, it is a False Positive (FP) also known as a Type I error
- 3) When the non-event values are predicted correct, it is True Positive (TN)
- 4) When the non-event values are predicted incorrect, it is a False Positive (FN) also known as a Type II error

The confusion matrix not only helps in describing the accuracy but is also very useful in measuring important parameters like Precision, Recall, and F-measure. TP_i , TN_i , FP_i , and FN_i represent real realism, real injustice, false opposition, and false falsehood, in the confusion matrix wrt i th class. The recall is represented by R and the accuracy of P .

Minimum includes working with the smallest possible unit (total facial images):

$$P_{micro} = \frac{\sum_{i=1}^{|G|} TP_i}{\sum_{i=1}^{|G|} TP_i + FP_i} (3)$$

$$R_{micro} = \frac{\sum_{i=1}^{|G|} TP_i}{\sum_{i=1}^{|G|} TP_i + FN_i} (4)$$

The micro F1-score is formulated using the relation of recall, micro-averaged precision, P_{micro} , and, R_{micro} :

$$F1_{micro} = 2 \cdot \frac{P_{micro} \cdot R_{micro}}{P_{micro} + R_{micro}} (5)$$

Assuming the divider gets a large [F1] micro, it works very fine. Here, a small amount may not be resistant to all speculative performance. Due to this, if there is an imbalance in class distribution, it can be misleading.

Macro Average is formulated as over large groups and over individual classroom performance rather than viewing by using:

$$P_{macro} = \frac{1}{|G|} \sum_{i=1}^{|G|} TP_i / TP_i + FP_i (6)$$

$$R_{macro} = \frac{1}{|G|} \sum_{i=1}^{|G|} TP_i / TP_i + FN_i (7)$$

Macro F1-score is formulated as:

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

$$F1_{macro} = 2 \cdot \frac{P_{macro} R_{macro}}{P_{macro} + R_{macro}} \quad (8)$$

If the value of $\square 1 \square \square \square \square$ is bigger, then it indicated that the classifier performs for individual classes.

1 Multi-class accuracy is defined as the average of the correct predictions and formulated as:

$$accuracy = \frac{1}{N} \sum_{k=1}^{|G|} \sum_{x: g(x)=k} I(g(x) = \hat{g}(x)) \quad (9)$$

10 Where, the indicator function is represented, which returns 1 when there is any match between the classes and otherwise it will return 0.

3 VGG16 has several differences from AlexNet. Instead of using large receptive fields like Alexnet, VGG16 uses small receptive fields which allows VGG16 to have a large number of weight layers. This result helps VGG16 to achieve higher accuracy but also makes the training process more compute and time-consuming. The accuracy of VGG 16 is generally seen to be 7% - 10% higher than AlexNet.

5. FUTURE WORK

1 The proposed Facial Recognition System is based on the concept of Transfer Learning and the VGG16 Neural Network. It consists of five main Stages: (i) Data Collection, (ii) Data purification (Data cleaning), Fine-tuning of the VGG16 model, (iv) Model Training, and (v) Performance Checking. As we already discussed how this fine-tuned model works better for this particular situation although the dataset was very small. The model can be tested with much bigger datasets i.e. BigData to get a better understanding of how this model will perform in real-life situations. The transfer learning i.e. fine-tuning methodology should not only be limited to VGG16. There are various Neural Networks like ResNet, Inception, etc. which perform better than VGG16. Hyperparameters can also be adjusted accordingly to get desired results under exceptional conditions like position scale, camera angle variations, blurred vision, extremely low light, extremely bright light, etc.

With help of this, big leaps of improvement can be achieved. The Artificial Intelligence industry is growing very rapidly and is extensively used almost everywhere in the world. This will take the AI in mobile phone cameras, self-driving cars, Robotic Process Automation (RPA), Healthcare Automation, Social Media platforms, and Security systems to the very next level.

REFERENCES

- [1] Saypadith S. and S. Aramvith (2018), "Real-Time Multiple Face Recognition using Deep Learning on Embedded GPU System", Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), Pp.1318-1324, 2018.

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

- [2] Chetna Singh, Prashant Baheti, and Sarvesh Singh, "Human Face Recognition and Face Detection using Skin Colour Model", Researchgate, 2014.
- [3] Shubha P. and Meenakshi M., "Human Face Recognition Using Local Binary Pattern Algorithm Real-Time Validation", International Conference On Computational Vision and Bio-Inspired Computing, Pp.240-246, 2019.
- [4] KH Teoh, RC Ismail, SZM Naziri, R Hussin, MNM Isa, and MSSM Basir, "Face Recognition and Identification using Deep Learning Approach", 2021.
- [5] Suleman Khan, Shah, Ehtasham Ahmed, Syed A. A. M. Hammad Javed, and Syed Umaid Ali, "Transfer Learning of a Neural Network Using Deep Learning to Perform Face Recognition", International Conference on 24 - 25 July 2019, IEEE.
- [6] Narayan T. Deshpande and Dr. S. Ravishankar (2017), "Face Detection and Recognition using Viola-Jones algorithm and Fusion of PCA and ANN".
- [7] Sun Y., X. Tang, and X. Wang, "Deep learning face representation from predicting 10000 classes", Proc. IEEE Conference Computer Vision Pattern Recognition, pp. 1891-1898, Jun. 2014.
- [8] Manik Sharma, J Anuradha, H KManne and G S C Kashyap (2017), "Facial Detection using Deep Learning", IOP Conference Series Materials Science and Engineering 263(4):042092.
- [9] Deepesh Raj, "A real-time Face Recognition System using PCA and various distance classifiers", CS676: Computer Vision and Image Processing, Pp.1-11, 2011.
- [10] Showkat A. Dara, S. Palanivel, "Neural Networks (CNNs) and VGG on Real-Time Face Recognition System", Turkish Journal of Computer and Mathematics Education Vol.12 No.9 (2021), 1809-1822.
- [11] S. SwethaK, E. Surekha, S. Sindhuja, Rohini Kailas, "Attendance Management using face detection and recognition", 2020.
- [12] P. Apoorva, H. C. Impana, S. Siri, M. Varshitha., B. Ramesh, "Identification by Face Recognition using Open Computer Vision Classifiers", IEEE 3rd International Conference on Computing Methodologies and Communication (ICCMC), Pp.775-778, 2019.
- [13] K. Jaya Prasanna Lakshmi, Sridevi Warriar, and T. Kumar, "Automated Face Recognition by Smart Security System Using AI & ML Algorithms", Published in 2021 5th International Conference on Trends in Electronics and Informatics (ICOEI), IEEE.
- [14] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, "Deep Face Recognition", In BMVC, vol. 1, p. 6 (2015).

In this proposed methodology, the model training time was seen to reduce quite dramatically than training a VGG16 model. And the model accuracy seems to increase by 8% - 9% as compared to the VGG16 model. Although the dataset on which the models were tested was very small, the initial impressions are pretty good and promising. This proposed model can further be checked on large datasets to get a better understanding of the accuracy. And there was no case of overfitting seen in the model.

AI RESEARCH PAPER (Updated).docx

ORIGINALITY REPORT

9%

SIMILARITY INDEX

PRIMARY SOURCES

1	turcomat.org Internet	55 words — 1%
2	www.evbuild.com Internet	41 words — 1%
3	towardsdatascience.com Internet	40 words — 1%
4	www.semanticscholar.org Internet	29 words — 1%
5	ebin.pub Internet	26 words — < 1%
6	www.hindawi.com Internet	23 words — < 1%
7	"Proceedings of the International Conference on Artificial Intelligence and Computer Vision (AICV2020)", Springer Science and Business Media LLC, 2020 Crossref	20 words — < 1%
8	"Sustainable Communication Networks and Application", Springer Science and Business Media LLC, 2022 Crossref	16 words — < 1%

9	"Neural Information Processing", Springer Science and Business Media LLC, 2017 Crossref	15 words — < 1%
10	www.astesj.com Internet	15 words — < 1%
11	www.ijert.org Internet	15 words — < 1%
12	medium.com Internet	14 words — < 1%
13	R L Priya, Shreeja Nanda, Maitraiya Dandekar, Anmol Devnani, Riteshsingh Kadakoti. "Face-Recognition Based Attendance System using One-Shot Learning", 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), 2021 Crossref	13 words — < 1%
14	Yanan Zhang, Hongyu Wang, Fang Xu, Kai Jia. "A Deep Face Recognition Method Based on Model Fine-tuning and Principal Component Analysis", 2017 IEEE 7th Annual International Conference on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER), 2017 Crossref	13 words — < 1%
15	www.ijarse.com Internet	13 words — < 1%
16	"Computational Vision and Bio-Inspired Computing", Springer Science and Business Media LLC, 2020 Crossref	11 words — < 1%
17	Lopez, Amy E.. "The Use of Facial Recognition Technology by the United States and China, and	10 words — < 1%

How It Affects Civil Liberties", Utica College, 2020

ProQuest

-
- | | | |
|----|---|-----------------|
| 18 | deepai.org
Internet | 10 words — < 1% |
|----|---|-----------------|
-
- | | | |
|----|---|----------------|
| 19 | Dinesh Jackson Samuel R, Rajesh Kanna B.
"Cybernetic microbial detection system using
transfer learning", Multimedia Tools and Applications, 2018
Crossref | 9 words — < 1% |
|----|---|----------------|
-
- | | | |
|----|--|----------------|
| 20 | Omkar Mule, Abhijeet Pandit, Sujoy Karmakar,
Atharva Kavale, Mr. Uttam Waghmode.
"Supervisory framework for threat detection with multilayer
processing in CNN", ITM Web of Conferences, 2022
Crossref | 9 words — < 1% |
|----|--|----------------|
-
- | | | |
|----|--|----------------|
| 21 | Showkat Ahmad Dar, S Palanivel. "Performance
Evaluation of Convolutional Neural Networks
(CNNs) And VGG on Real Time Face Recognition System",
Advances in Science, Technology and Engineering Systems
Journal, 2021
Crossref | 9 words — < 1% |
|----|--|----------------|
-
- | | | |
|----|---|----------------|
| 22 | ijsrcseit.com
Internet | 9 words — < 1% |
|----|---|----------------|
-
- | | | |
|----|---|----------------|
| 23 | www.ai4europe.eu
Internet | 9 words — < 1% |
|----|---|----------------|
-
- | | | |
|----|---|----------------|
| 24 | www.globalscientificjournal.com
Internet | 9 words — < 1% |
|----|---|----------------|
-
- | | | |
|----|---|----------------|
| 25 | academic-accelerator.com
Internet | 8 words — < 1% |
|----|---|----------------|
-
- | | | |
|----|---|----------------|
| 26 | docplayer.net
Internet | 8 words — < 1% |
|----|---|----------------|

-
- 27 openreview.net 8 words — < 1%
Internet
-
- 28 repository.iiitd.edu.in 8 words — < 1%
Internet
-
- 29 www.repo.uni-hannover.de 8 words — < 1%
Internet
-
- 30 "Evolving Technologies for Computing, Communication and Smart World", Springer Science and Business Media LLC, 2021 7 words — < 1%
Crossref
-
- 31 Soham Wattamwar, Rishikesh Mate, Parth Rainchwar, Shamla Mantri, Geeta Sorate. "Optimal Face Recognition System using Haar Classifier", 2021 International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON), 2021 7 words — < 1%
Crossref
-
- 32 "Proceedings of the 21st EANN (Engineering Applications of Neural Networks) 2020 Conference", Springer Science and Business Media LLC, 2020 6 words — < 1%
Crossref
-
- 33 Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, L. D. Jackel. "Backpropagation Applied to Handwritten Zip Code Recognition", Neural Computation, 1989 6 words — < 1%
Crossref
-

